Breaking the Speed Limit

Wenxin Jiang **Jian Yin**

Department of Biostatistics, City University of Hong Kong PyCon US 2026

CityU 香港城市大學 City University of Hong Kong

Thesis

Simulation is how statisticians write tests when real data has no answer key.

Laptop defines trust. **Server CPU** tests scale. **A100** matters only after the workload becomes GPU-shaped.

Simulation contract

- Define the statistic and null behavior.
- Keep a readable reference as the oracle.
- Test null, alternative, and pathological scenarios.
- Scale only after equivalence, calibration, and recovery pass.

Six-step workflow

Target statistic
↓
Reference oracle
↓
Scenario grid
↓
Validation
↓
Scale
↓
Optimize

Three failure modes

Risk	Signal
Implementation	Optimized path changes the statistic.
Statistical	Null p-values drift from calibration.
Systems	Runtime improves while memory keeps growing.

MacBook Air: trust tier

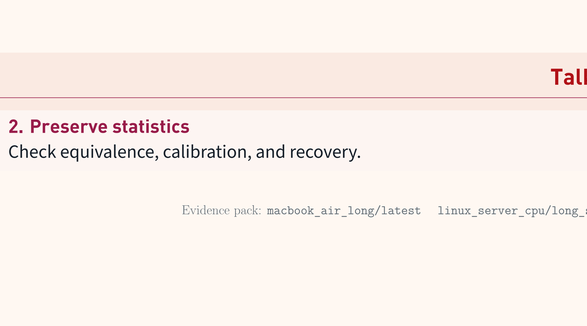
3,840 k-means pass rows

450 perm pass rows

0 local fail rows

This tier is for correctness, debugging, and scenario coverage, not final hardware speed claims.

k-means failure surface



ARI = adjusted rand index. Runtime matters only after recovery is visible.

Reference preservation

- k-means optimized paths preserve inertia against the reference.
- permutation matrix paths preserve p-values against the loop.
- memory-risk skips are explicit rows, not hidden omissions.

Server CPU and A100: scale tier

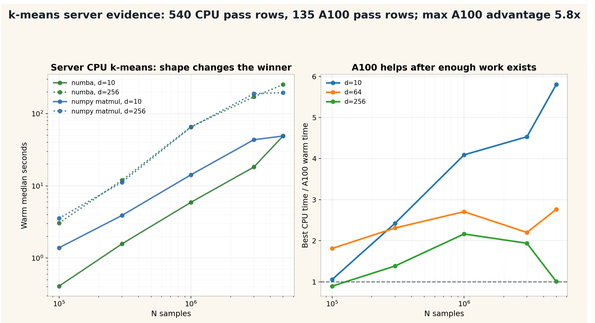
540 CPU k-means rows

135 A100 rows

5.8x A100 edge

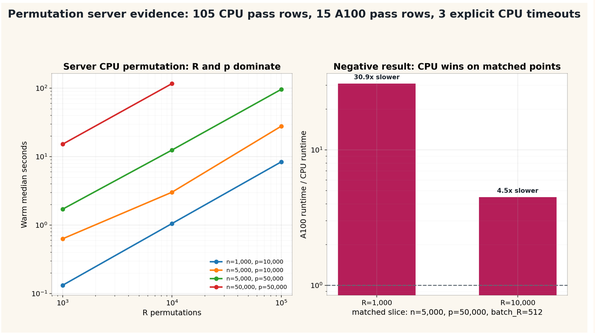
Server results are interpreted as the next rung in the evidence ladder.

k-means: shape-dependent acceleration



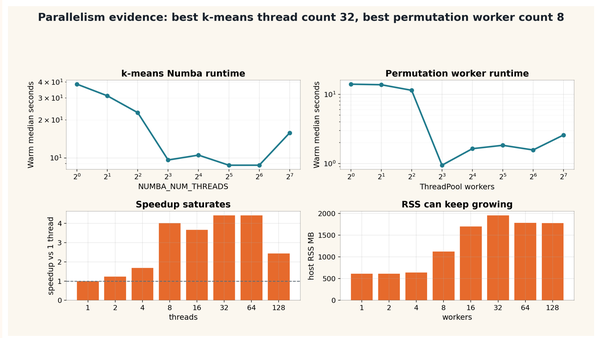
A100 helps only after enough regular array work exists.

Permutation: useful negative result



Validated negative evidence: the matched A100 permutation path loses to CPU here.

Parallelism must be tuned



Best speed was not at maximum parallelism; memory can keep growing after runtime stops improving.

Memorable findings

Validate before optimizing

Faster code that changes the statistic is a different method.

Shape beats logo

Numba, NumPy matmul, threads, and A100 each win on specific shapes.

Negative GPU evidence is useful

It separates a valid formulation from a winning accelerator path.

Decision guide

- Start with the clearest reference.
- Use Numba for hot scalar CPU loops.
- Use algebra before creating giant temporaries.
- Treat GPU as a formulation change.

Clear enough to trust. Fast enough to scale.

Evidence and slides

`github.com/lucajiang/FastStatisticalModels4Python`

Talk spine

1. Simulate truth
Build scenarios with known behavior.

2. Preserve statistics
Check equivalence, calibration, and recovery.

3. Scale evidence
Move laptop to server CPU to A100.

4. Pick the smallest tool
Pick the tool that removes the proven bottleneck.

Evidence pack: `macbook_air_long/latest` `linux_server_cpu/long_safe_20260503_190133` `linux_server_a100/long_safe_20260503_190133`